

Project Phase 1: Data Requirements

Pokémon Battle League Database System

TEAM No: 58

Student Name: M M V Santosh Reddy

Roll No: 2024101067

Student Name: B Nikhil Mohan

Roll No: 2024101010

Student Name: K Tilak choudary

Roll No: 2024101061

1. Mini World Development

1.1 Idea Formulation

Fantasy World: Pokémon Universe - Regional Battle League System

Concept Overview:

A database system managing the Pokémon Battle League across regions (Kanto, Johto, Hoenn). The system tracks trainers, Pokémon teams, gym challenges, tournaments, item inventories, and trainer friendships. This focuses on competitive infrastructure supporting trainers, gym leaders, and league officials.

Database System Integration:

1. **Trainer Management:** Track registration, badges, experience levels, and rankings
2. **Pokémon Tracking:** Manage ownership, evolution conditions, stats, and battle readiness
3. **Gym System:** Coordinate challenges, badge awards, and leader statistics
4. **Tournament Operations:** Organize brackets, track results, calculate rankings
5. **Item Economy:** Manage evolution stones, TMs, potions, and held items
6. **Friendship Network:** Enable trainer connections for trading and battles
7. **Battle Analytics:** Record battle histories and performance metrics

Novelty Factor:

The system handles **cascading competitive mechanics**: when a trainer defeats a gym leader, the database automatically updates badges, triggers evolution checks, awards items, unlocks tournaments, and updates leaderboards. It manages **complex evolution**

conditions (experience + friendship + items + time + location) across 800+ Pokémon species. The system ensures **real-time tournament integrity** through validation, fraud prevention, and atomic transaction processing.

1.2 User Identification & Interaction

Primary Users:

1. Pokémon Trainers

- View team stats, evolution readiness, battle history
- Track badge progress, schedule gym challenges
- Check item inventory and tournament eligibility
- **Interaction:** PokéDex terminals, Trainer Card apps
- **Frequency:** Daily during active gameplay

2. Gym Leaders

- Review challenger profiles and team compositions
- Award badges to victorious trainers
- Analyze gym statistics and win rates
- **Interaction:** Leader Console at gyms
- **Frequency:** Daily during operations

3. League Officials

- Generate championship reports
- Manage tournaments and validate entries
- Verify badge authenticity
- **Interaction:** League Management Portal
- **Frequency:** Daily monitoring, weekly reports

4. Pokémon Center Nurses

- Update Pokémon health status
- Manage Storage System deposits/withdrawals
- **Interaction:** Center healing stations
- **Frequency:** Continuous operations

5. Tournament Coordinators

- Organize events, manage registrations
- Track match results and update brackets
- **Interaction:** Tournament systems
- **Frequency:** During tournament seasons

6. Item Shop Vendors

- Monitor stock levels, process purchases
- Adjust pricing based on demand
- **Interaction:** Shop inventory systems
- **Frequency:** Daily operations

7. Pokémon Researchers

- Analyze evolution patterns, battle effectiveness
- Study type matchup statistics
- **Interaction:** Research analytics platforms
- **Frequency:** Weekly/monthly analysis

1.3 Purpose of the Database

Critical Functions:

1. **Complex Relationship Management:** Handles trainers owning multiple Pokémon, evolution chains requiring items AND friendship, gym battles with 6-member teams, and tournament brackets. Spreadsheets require manual updates across dozens of cells for one battle.
2. **Atomic Transaction Integrity:** When defeating a gym leader, atomically updates badges, experience, battle history, evolution checks, and rankings. If any step fails, ALL changes rollback. Spreadsheets offer zero protection.
3. **Fraud Prevention:** Prevents fake badges (referential integrity), invalid tournament entries (check constraints), and impossible results (triggers). Spreadsheets allow unlimited manipulation.
4. **Massive Scalability:** Handles 50,000+ trainers per region, 500+ gyms, 10,000+ daily battles. Database indexes enable sub-second queries; spreadsheets crash with large data.
5. **Dynamic Evolution Rules:** Manages complex conditions (experience + friendship + items + time + location) across 800+ species automatically.
6. **Advanced Analytics:** Generates strategic reports requiring multi-table joins in milliseconds.
7. **Concurrent Access:** 1,000+ trainers battle simultaneously. Database ensures consistency; spreadsheet file locking blocks concurrent users.

Why Database Over Alternatives:

- **Spreadsheets:** Cannot enforce rules, merge conflicts, row limits, no referential integrity
- **Paper Records:** No real-time sync, prone to loss, impossible queries

- **Simple Apps:** No central authority, poor concurrency, manual constraints

The database provides ACID properties, automated enforcement, and reliable concurrent access.

2. Database Requirements

2.1 Entity Types

STRONG ENTITIES (6):

1. TRAINER

- TrainerID (PK)
- RegistrationNumber (Alternate Key)
- TrainerName, Hometown, Region
- RegistrationDate, ExperienceLevel
- TotalBadges, ContactEmail

2. POKEMON

- PokemonID (PK)
- Nickname (Alternate Key within trainer)
- SpeciesName, PrimaryType, SecondaryType
- Level, ExperiencePoints, MaxHP, CurrentHP
- IsShiny, CatchDate, FriendshipLevel

3. GYM

- GymID (PK)
- GymName, City, Region
- TypeSpecialty, BadgeName
- DifficultyLevel

4. GYM_LEADER

- LeaderID (PK)
- LeaderName, SpecializationType
- YearsActive, SignaturePokemon
- LeaderRank

5. TOURNAMENT

- TournamentID (PK)
- TournamentName, TournamentRegion

- StartDate, EndDate
- EntryBadgeRequirement
- TournamentStatus, MaxParticipants

6. ITEM

- ItemID (PK)
- ItemName, ItemType
- RarityLevel, PurchaseCost
- EffectDescription

WEAK ENTITIES (2):

7. BATTLE_RECORD (depends on TRAINER)

- BattleID (Partial Key)
- TrainerID (FK)
- BattleDate, OpponentType
- BattleOutcome, ExperienceGained

8. POKEMON_MOVE (depends on POKEMON)

- MoveSlotNumber (Partial Key, 1-4)
- PokemonID (FK)
- MoveName, MoveType, MovePower

2.2 Entities with Multi-Key Attributes

TRAINER:

- Primary Key: TrainerID
- Alternate Key: RegistrationNumber

POKEMON:

- Primary Key: PokemonID
- Alternate Key: Nickname + TrainerID

2.3 Weak Entity Types

1. BATTLE_RECORD

- Owner: TRAINER
- Complete Key: TrainerID + BattleID
- Business Rule: Every battle record belongs to exactly one trainer

2. POKEMON_MOVE

- Owner: POKEMON
- Complete Key: PokemonID + MoveSlotNumber
- Business Rule: Max 4 moves per Pokémon

2.4 Relationship Types (8 Total)

R1: OWNS (TRAINER owns POKEMON)

- Cardinality: 1:N
- Participation: TRAINER (Partial), POKEMON (Total)
- Attributes: AcquisitionDate, CatchLocation, IsInActiveParty
- Constraint: Max 6 Pokémon with IsInActiveParty = TRUE

R2: CHALLENGES (TRAINER challenges GYM)

- Cardinality: M:N
- Participation: Both Partial
- Attributes: ChallengeDate, ChallengeResult, AttemptsCount, BadgeEarned

R3: LEADS (GYM_LEADER leads GYM)

- Cardinality: 1:1
- Participation: Both Total
- Attributes: AppointmentDate, TenureYears

R4: PARTICIPATES_IN (TRAINER participates in TOURNAMENT)

- Cardinality: M:N
- Participation: Both Partial
- Attributes: RegistrationDate, PlacementRank, RoundsWon, IsEliminated

R5: CARRIES (TRAINER carries ITEM)

- Cardinality: M:N
- Participation: Both Partial
- Attributes: QuantityOwned, AcquisitionDate

R6: BEFRIENDS (TRAINER befriends TRAINER) - Unary

- Cardinality: M:N
- Participation: Partial
- Attributes: FriendshipDate, FriendshipLevel, TradeCount

R7: HOLDS (POKEMON holds ITEM)

- Cardinality: M:1
- Participation: Both Partial
- Attributes: EquippedDate, BoostPercentage

R8: EVOLVES_INTRO (POKEMON evolves into POKEMON) - Unary

- Cardinality: 1:1
- Participation: Partial
- Attributes: EvolutionTrigger, RequiredLevel, RequiredItem
- Constraint: No circular evolution chains

2.5 Relationships with $n > 3$ Participants

Ternary: BATTLE_PARTICIPATION

- Entities: TRAINER $\times$ POKEMON $\times$ GYM
- Attributes: BattleDate, PokemonUsed, BattleResult, TotalTurns
- Business Rule: Tracks which Pokémon used in specific gym battles

Quaternary: TRAINING_SESSION

- Entities: TRAINER $\times$ POKEMON $\times$ ITEM $\times$ LOCATION
- Attributes: SessionDate, DurationMinutes, SuccessRate
- Business Rule: Tracks where trainers used items to train Pokémon

2.6 Subclass Hierarchy

POKEMON (Superclass)

Subclasses:

- **LEGENDARY_POKEMON** (Disjoint)
 - LegendaryTier, CaptureRateModifier, SpecialAbility
- **STARTER_POKEMON** (Disjoint)
 - StarterGeneration, RegionOrigin, EvolutionStage

Characteristics: Disjoint, Partial

Business Rule: Legendary Pokémon have stricter capture conditions

2.7 Composite, Multi-valued, and Derived Attributes

Composite Attributes:

1. TRAINER.ContactInfo: {ContactEmail, PhoneNumber}
2. GYM.LocationDetails: {City, Region, Coordinates}
3. TOURNAMENT.DateRange: {StartDate, EndDate, DurationDays}

Multi-valued Attributes:

1. POKEMON.TypeList: {PrimaryType, SecondaryType}
2. TRAINER.BadgeCollection: Stored in CHALLENGES table

3. POKEMON.MoveSet: Up to 4 moves in POKEMON_MOVE table

Derived Attributes:

1. TRAINER.WinRate: $(\text{TotalWins} / (\text{TotalWins} + \text{TotalLosses})) \times 100$
2. POKEMON.NextEvolutionXP: $\text{RequiredXP} - \text{CurrentExperiencePoints}$
3. TOURNAMENT.ParticipationCount: $\text{COUNT}(\text{Trainers in PARTICIPATES_IN})$
4. GYM.ChallengeSuccessRate: $(\text{Successful} / \text{Total}) \times 100$

2.8 Constraints and Business Rules

Domain Constraints:

- TRAINER.ExperienceLevel $\in \{\text{'Novice'}, \text{'Intermediate'}, \text{'Advanced'}, \text{'Expert'}, \text{'Master'}\}$
- POKEMON.Level $\in [1, 100]$
- POKEMON.FriendshipLevel $\in [0, 255]$
- TRAINER.TotalBadges $\in [0, 8]$
- POKEMON.CurrentHP $\leq \text{MaxHP}$

Cardinality Constraints:

- Max 6 Pokémon with IsInActiveParty = TRUE per trainer
- Max 4 moves per Pokémon
- Each trainer earns each gym badge only once
- Each gym has exactly 1 gym leader

Participation Constraints:

- Every POKEMON must be owned by exactly one TRAINER (total)
- Every GYM must be led by exactly one GYM_LEADER (total)
- Every BATTLE_RECORD belongs to a TRAINER (identifying)

Integrity Constraints:

1. Pokémon with CurrentHP = 0 cannot battle
2. Tournament entry requires TotalBadges $\geq$ EntryBadgeRequirement
3. BadgeEarned = TRUE only if ChallengeResult = 'Victory'
4. No circular evolution chains
5. Tournament.EndDate $\geq$ StartDate

Referential Integrity:

- CASCADE on DELETE: Deleting TRAINER removes owned POKEMON, BATTLE_RECORDS
- RESTRICT on DELETE: Cannot delete GYM_LEADER if leading a gym
- SET NULL: BEFRIENDS relationships set to null

3. Functional Requirements

3.1 Retrieval Operations

3.1.1 Selection Operations

Query 1: Find Fire-type Pokémon from Pallet Town trainers above level 30

Purpose: Locate strong regional Fire-types for gym battles

Query 2: List all Legendary Pokémon currently available

Purpose: Help trainers locate rare Pokémon

Query 3: Retrieve trainers eligible for Elite Four (8 badges)

Purpose: Identify championship-qualified trainers

3.1.2 Projection Operations

Query 1: Display names, species, and levels of active party

Purpose: Show essential battle information

Query 2: List gym names and specialty types

Purpose: Quick gym challenge overview

Query 3: Show tournament names and entry requirements

Purpose: Find eligible tournaments

3.1.3 Aggregate Operations

Query 1: Calculate average Pokémon level by species

Purpose: Analyze training popularity

Query 2: Find gym with highest challenge success rate

Purpose: Identify most difficult gyms

Query 3: Sum experience points by region

Purpose: Compare regional competitiveness

Query 4: Calculate maximum level per Pokémon type

Purpose: Identify well-trained types

3.1.4 Search Operations

Query 1: Partial text matching for trainer names (e.g., "Ash")

Purpose: Flexible trainer search

Query 2: Find Pokémon species by partial name (e.g., "char" matches Charizard)

Purpose: Locate species with incomplete info

Query 3: Search items with effect keywords (e.g., "evolution")

Purpose: Find specific items for evolution

Query 4: Locate gyms by city/region partial match

Purpose: Find nearby gyms

3.2 Analysis Reports (Minimum 3)

Report 1: Regional Trainer Performance Comparison

Compares win rates, average Pokémon levels, and badge rates across regions.

Join: TRAINER × OWNS × POKEMON × CHALLENGES × GYM

Metrics: Average team level, badges per trainer, regional win rate, dominant types

Insight: Identify underperforming regions needing resources; help trainers assess competition

Report 2: Gym Challenge Difficulty Analysis

Evaluates gym balance through success rates and attempt counts.

Join: GYM × CHALLENGES × TRAINER × POKEMON

Metrics: Success rate, average attempts, challenger level at victory, type effectiveness

Insight: Identify bottleneck gyms; rebalance difficulty; guide optimal challenge sequences

Report 3: Tournament Meta-Game Analysis

Reveals winning strategies by analyzing type combinations and team structures.

Join: TOURNAMENT × PARTICIPATES_IN × TRAINER × OWNS × POKEMON × HOLDS × ITEM

Filter: PlacementRank ≤ 3

Metrics: Most-used species, winning type combos, effective held items, winner vs loser levels

Insight: Optimize team building; identify overpowered combinations; balance adjustments

Report 4: Item Economy and Evolution Demand

Tracks item circulation and evolution stone demand.

Join: ITEM × CARRIES × TRAINER × OWNS × POKEMON

Metrics: Items in circulation, evolution stone usage, regional availability, restock priority

Insight: Optimize shop inventory; adjust drop rates; identify economic imbalances

3.3 Modification Operations

(a) Insertion with Constraint Checking

Operation 1: Register new trainer

Validation: Unique RegistrationNumber, TotalBadges = 0, valid city, email format

Purpose: Prevent duplicate registrations, ensure data integrity

Operation 2: Capture new Pokémon

Validation: TrainerID exists, team size < 6, Level 1-100, CurrentHP ≤ MaxHP, unique nickname

Purpose: Maintain team limits, ensure valid ownership

Operation 3: Register for tournament

Validation: Tournament active, TotalBadges ≥ requirement, not already registered, has active party

Purpose: Enforce eligibility, prevent over-registration

(b) Update with Cascading Business Rules

Operation 1: Record gym victory

Cascade: Update TotalBadges, validate sequential order, award badge item, update gym stats, check championship eligibility

Purpose: Maintain badge integrity, enforce progression

Operation 2: Level up Pokémon

Cascade: Check evolution eligibility, verify item requirements, validate CurrentHP ≤ MaxHP, alert trainer

Purpose: Automate evolution logic, maintain stat relationships

Operation 3: Update tournament results

Cascade: Recalculate all participant ranks, update experience level, check elimination, award prizes, update leaderboards

Purpose: Maintain real-time rankings, automate prizes

Operation 4: Increase friendship after battle

Cascade: Check friendship-based evolution, unlock special moves

Purpose: Track relationship progression, trigger mechanics

(c) Deletion with Referential Integrity

Operation 1: Remove inactive trainer

Cascade: Delete owned Pokémon, battle records, challenges, tournament entries, item inventory

Set Null: Befriends relationships

Archive: Move to TrainerArchive

Purpose: Maintain consistency, prevent orphaned records

Operation 2: Delete completed tournament

Cascade: Remove ParticipatesIn records, bracket entries

Restrict: Cannot delete if active

Archive: Transfer to TournamentHistory

Purpose: Database maintenance, preserve historical data

Operation 3: Remove gym leader

Restrict: Cannot delete if leading gym

Required: Assign new leader first, validate qualifications, update appointment date

Purpose: Ensure continuous operations, prevent orphaned gyms

Operation 4: Release Pokémon

Cascade: Delete PokemonMove records, release held item

Check: Cannot release if only active party member

Archive: Record in ReleaseHistory

Purpose: Allow team management, prevent accidental releases